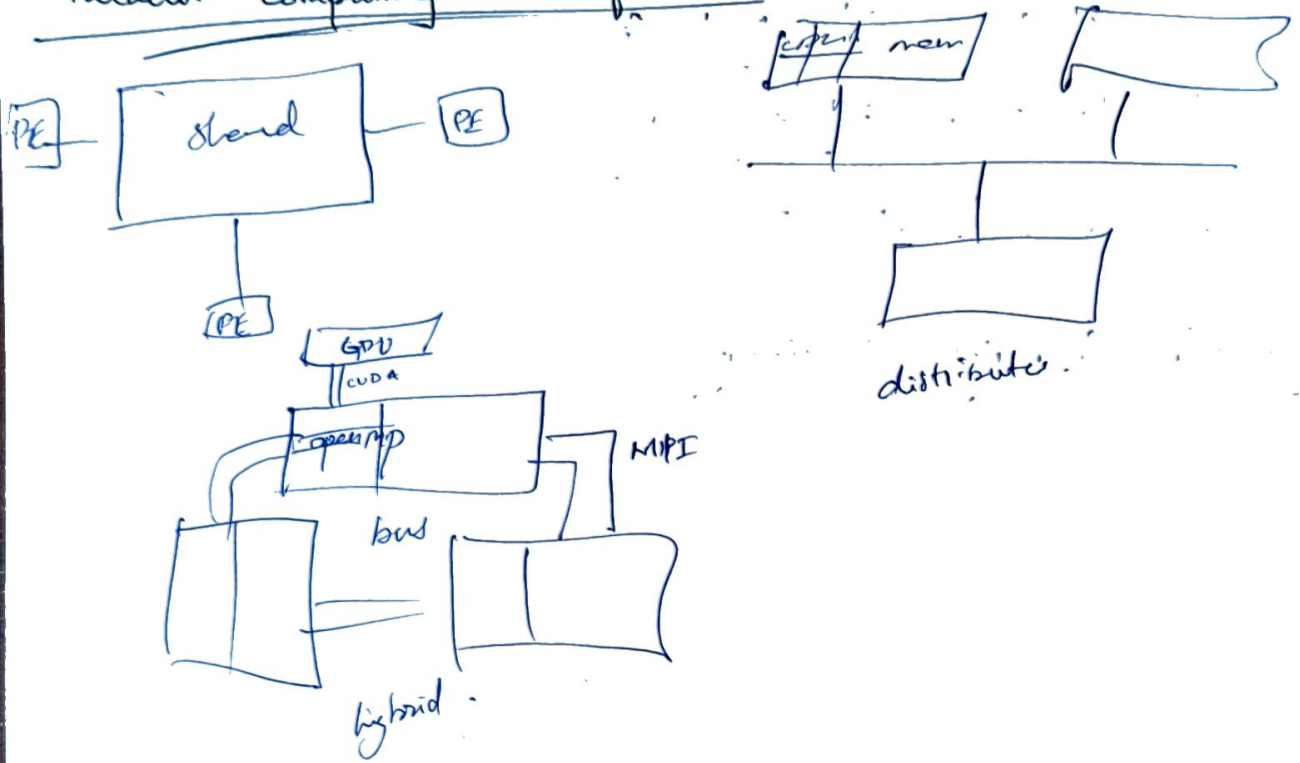
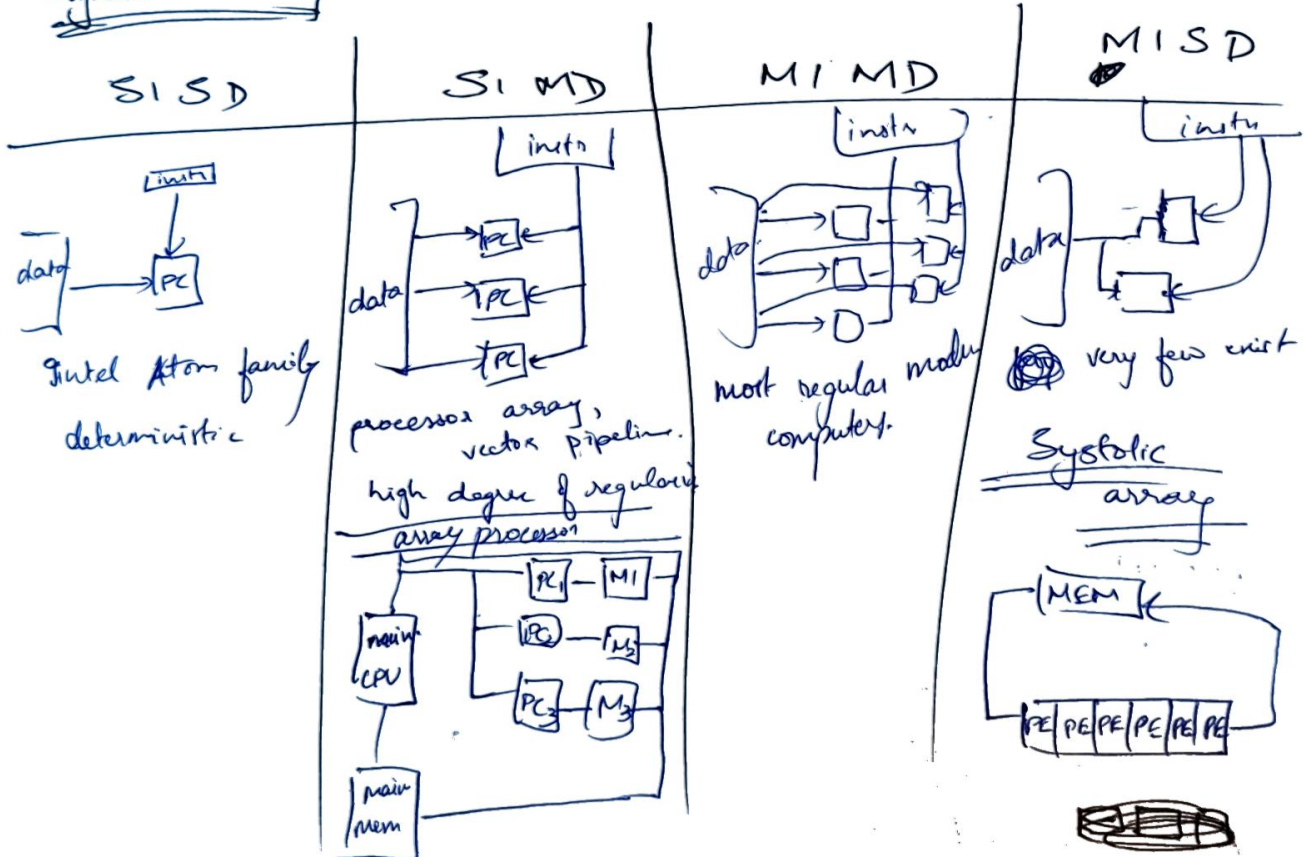
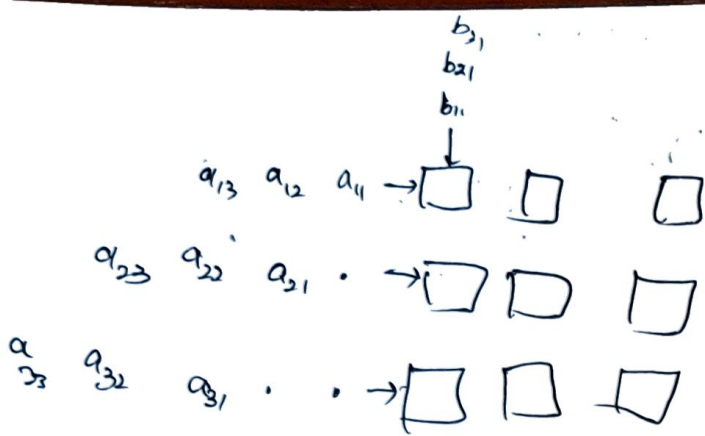


Parallel Computing Classification



Flynn's Taxonomy





Parallel Computing Terminology

SMP: Symmetric Multiprocessor $\rightarrow$ shared memory, single address space, equal access to all resources.

Communications: data exchange.

Synchronization: establish points for tasks to wait

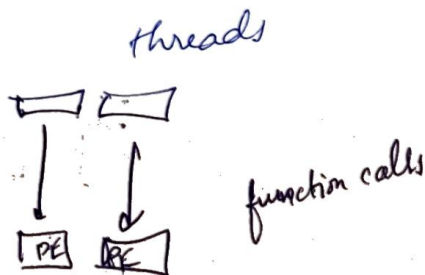
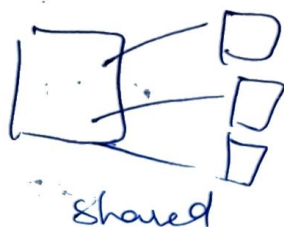
granularity = computation: communication

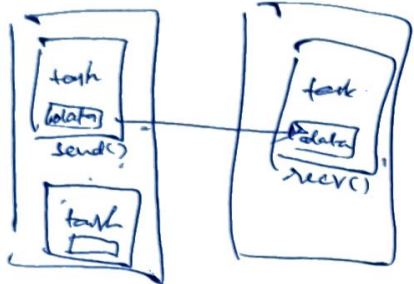
coarse
C.C.
fine

$$S = \frac{\text{wall clock serial}}{\text{wall clock parallel}}$$

time to
parallel overhead: coordinate tasks

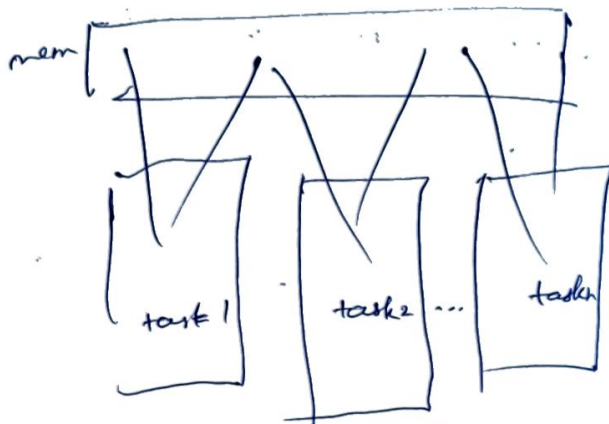
Models





distributed
cooperative operations

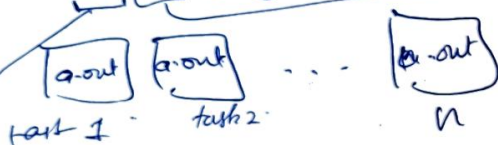
hybrid
OpenMP, MPI, CUDA
(gpu)
comp. intensive kernel



data parallel /
partitioned global address space
mode

set of tasks simultaneously
same operation
shared → everyone has access
distributed → split into chunks

SPMD (single prog multi data)



Same copy simultaneously
may use different data

tasks only execute
necessary portion.

~~MPI~~

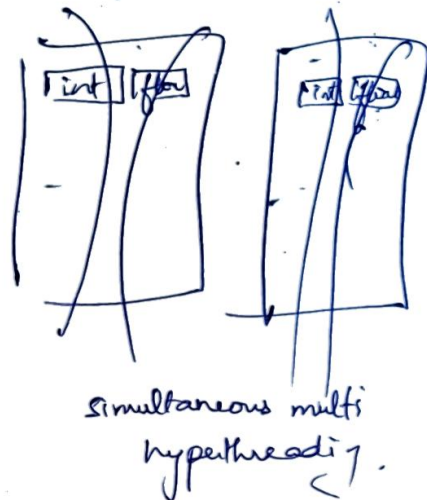
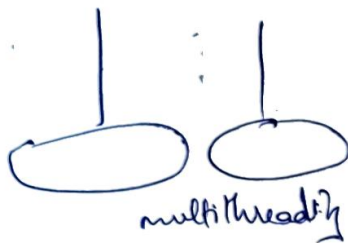
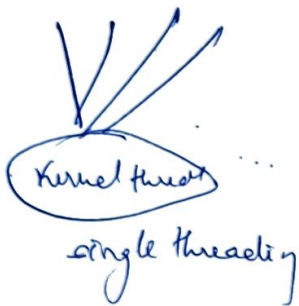
diff prog simultaneously

diff data
functional decomposition

parallelizing compiler

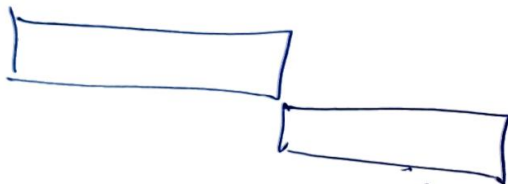
auto
programmer directed

Thread level parallelism



Instn. level

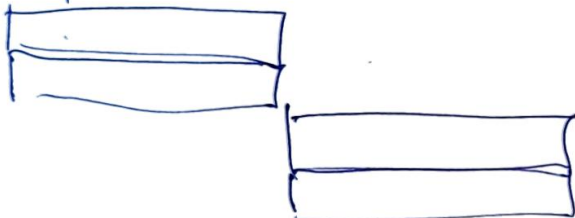
Scalar



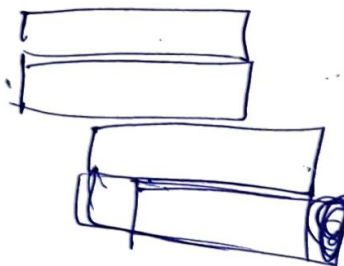
pipelining



superscalar



super pipelining



VLIW: Very Long Instruction Word

many independent instrs →

one word



is superscalar

static simple

dynamic scheduling
complex hardware

~~X~~
if no independent

instrs

insert NOPs

recompile

FALSE SHARING:

Core 1 uses X } on same cache line
" 2 uses Y }

for every change to X, full cache line update
unnecessary waste syncing.

OpenMP

~~1~~ #include <omp.h>

```
#pragma omp parallel  
{  
    omp_get_thread_num()  
}
```

gcc ~~for~~openmp test.c
export OMP_NUM_THREADS
= 4

./.out

#pragma omp parallel for : parallelizes for loops
collapse (2) : Nest 2 nested loops combined into
one large iteration space.

shared (var-name) : declares variable shared
among all threads.

#pragma omp critical { only one thread able to
update ~~at~~ ~~once~~ at once
var-name += 1
#pragma omp single : only one thread executes block
ONCE once-only
ops.

omp_set_num_threads(n) : no. of threads used
omp_get_num_threads() : default threads
omp_get_max_threads() : default threads

```
double start = omp_wtime()  
double end = omp_wtime()  
time Exec time = end - start
```

reduction(&sum) ^{variable}
gives each thread a copy
of the sum, combines safely
later

#pragma omp atomic : shared updates that very simple in
nature. low overhead compared to
critical. limited to specific expressions

Modern GPU

Streaming Multiprocessors

- contains several SPs (CUDA cores)
- shares resources (cub, regs, exec pipeline, cache)
- More SMs = more parallel computing power

Streaming Processors (CUDA core)

- execute individual threads
- More SPs = faster computations

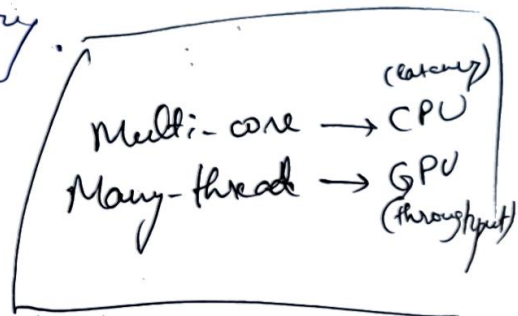
GDDR: Graphics Double Data Rate

SDRAM: Synchron. DRAM

HBM: High Bandwidth Memory

X: parallel

- ① efficient parallel algo
- ② mem access bottleneck
- ③ input sensitivity
- ④ mathematical recurrences



GPU Computing

thread → single line of exec.
warp → group of 32 threads
warp scheduler → managing/dispatching warps